Model Diagnostics and Visualization

GAM.jl Contributors

Table of contents

Introduction

This vignette demonstrates **gratia**-style model diagnostics and visualization tools available in GAM.jl. These functions mirror the R gratia package, providing tidy interfaces for inspecting fitted GAMs.

We fit a GAM to the classic Gu & Wahba (1991) simulated example with four smooth terms of varying complexity, then explore the full diagnostic toolkit.

Setup

```
using GAM
using StatsAPI: fitted, predict, residuals
using DataFrames
using Random
using Statistics
using Plots
```

Simulate Gu–Wahba example data

The test functions are:

- $f_0(x) = 2\sin(\pi x)$
- $f_1(x) = \exp(2x)$
- $f_2(x) = 0.2x^{11}(10(1-x))^6 + 10(10x)^3(1-x)^{10}$
- $f_3(x) = 0$ (null smooth)

```
Random.seed!(42)
n = 400

f0(x) = 2 * sin(  * x)
f1(x) = exp(2 * x)
f2(x) = 0.2 * x^11 * (10 * (1 - x))^6 + 10 * (10 * x)^3 * (1 - x)^10
f3(x) = 0.0

x0 = rand(n)
x1 = rand(n)
x2 = rand(n)
x3 = rand(n)

y = f0.(x0) .+ f1.(x1) .+ f2.(x2) .+ f3.(x3) .+ 2 .* randn(n)

df = DataFrame(y=y, x0=x0, x1=x1, x2=x2, x3=x3)
first(df, 5)
```

	y	x0	x1	x2	x3
	Float64	Float64	Float64	Float64	Float64
1	7.22842	0.173575	0.422258	0.553158	0.478497
2	5.76744	0.321662	0.737827	0.824626	0.221218
3	6.88207	0.258585	0.100835	0.314532	0.387745
4	3.85788	0.166439	0.321066	0.918532	0.535295
5	12.2319	0.527015	0.969515	0.732507	0.864098

Fit the GAM

```
m = gam(@gam_formula(y ~ s(x0, k=10, bs=:cr) + s(x1, k=10, bs=:cr) +
                         s(x2, k=10, bs=:cr) + s(x3, k=10, bs=:cr)), df)
```

```
Generalized Additive Model

Formula: y ~ 1

Family: Normal
Link:   IdentityLink
Method: REML

Parametric coefficients:

               Coef.  Std. Error      t  Pr(>|t|)

(Intercept)  7.75175    0.101379  76.46    <1e-99


Approximate significance of smooth terms:

Smooth                    edf   Ref.df

s(x0,bs=cr)              2.96        9
s(x1,bs=cr)              3.03        9
s(x2,bs=cr)              7.82        9
s(x3,bs=cr)              1.00        9


R² (adj) = 0.690   Deviance explained = 70.1%
Scale est. = 4.1111   n = 400
```

Model overview

The `overview` function provides a tidy summary of all smooth terms, including basis type, dimension, basis size, effective degrees of freedom (EDF), and EDF-to-k ratio.

```
ov = overview(m)
```

```
GAM Overview: 4 smooth term(s)

Smooth               Type           Dim     k      EDF  k-ratio

s(x0,bs=cr)          CubicSpline      1     9     2.96    0.329
s(x1,bs=cr)          CubicSpline      1     9     3.03    0.336
s(x2,bs=cr)          CubicSpline      1     9     7.82    0.869
s(x3,bs=cr)          CubicSpline      1     9     1.00    0.111
```

Smooth estimates

`smooth_estimates` evaluates each smooth function on a regular grid with pointwise standard errors. This is the foundation for plotting smooth effects.

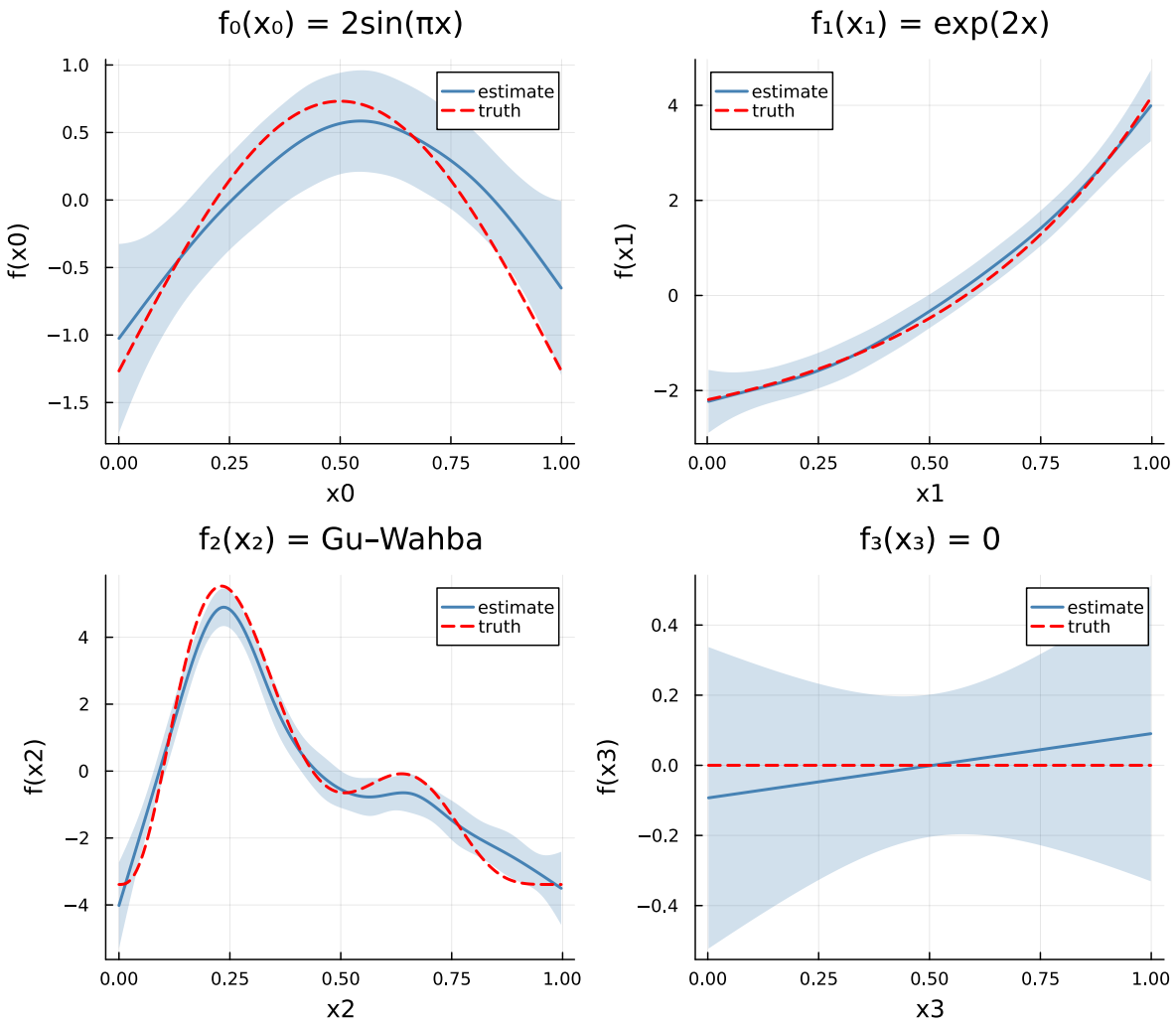
```
se = smooth_estimates(m; n=200)
```

```
SmoothEstimates: 4 smooth(s), 800 evaluation points
  s(x0,bs=cr): 200 points
  s(x1,bs=cr): 200 points
  s(x2,bs=cr): 200 points
  s(x3,bs=cr): 200 points
```

We plot each estimated smooth with ±2 SE confidence bands, overlaying the true function (centered to have zero mean, for identifiability):

```
true_fns = [f0, f1, f2, f3]
x_vars = [:x0, :x1, :x2, :x3]
x_data = [x0, x1, x2, x3]
labels = ["f (x ) = 2sin( x)", "f (x ) = exp(2x)", "f (x ) = Gu-Wahba", "f (x ) = 0"]

plots_se = []
for (i, xvar) in enumerate(x_vars)
```

```
    se_i = smooth_estimates(m; select=string(xvar), n=200)
    xgrid = se_i.covariates[xvar]
    est = se_i.estimate
    se_vals = se_i.se

    # True function on grid, centered
    f_grid = true_fns[i].(xgrid)
    f_grid = f_grid .- mean(f_grid)

    p = plot(xgrid, est;
        ribbon=2 .* se_vals, fillalpha=0.25, color=:steelblue,
        linewidth=2, label="estimate",
        xlabel=string(xvar), ylabel="f($(xvar))",
        title=labels[i])
    plot!(p, xgrid, f_grid;
        linestyle=:dash, linewidth=2, color=:red, label="truth")
    push!(plots_se, p)
end
plot(plots_se...; layout=(2, 2), size=(800, 700))
```



Select a specific smooth:

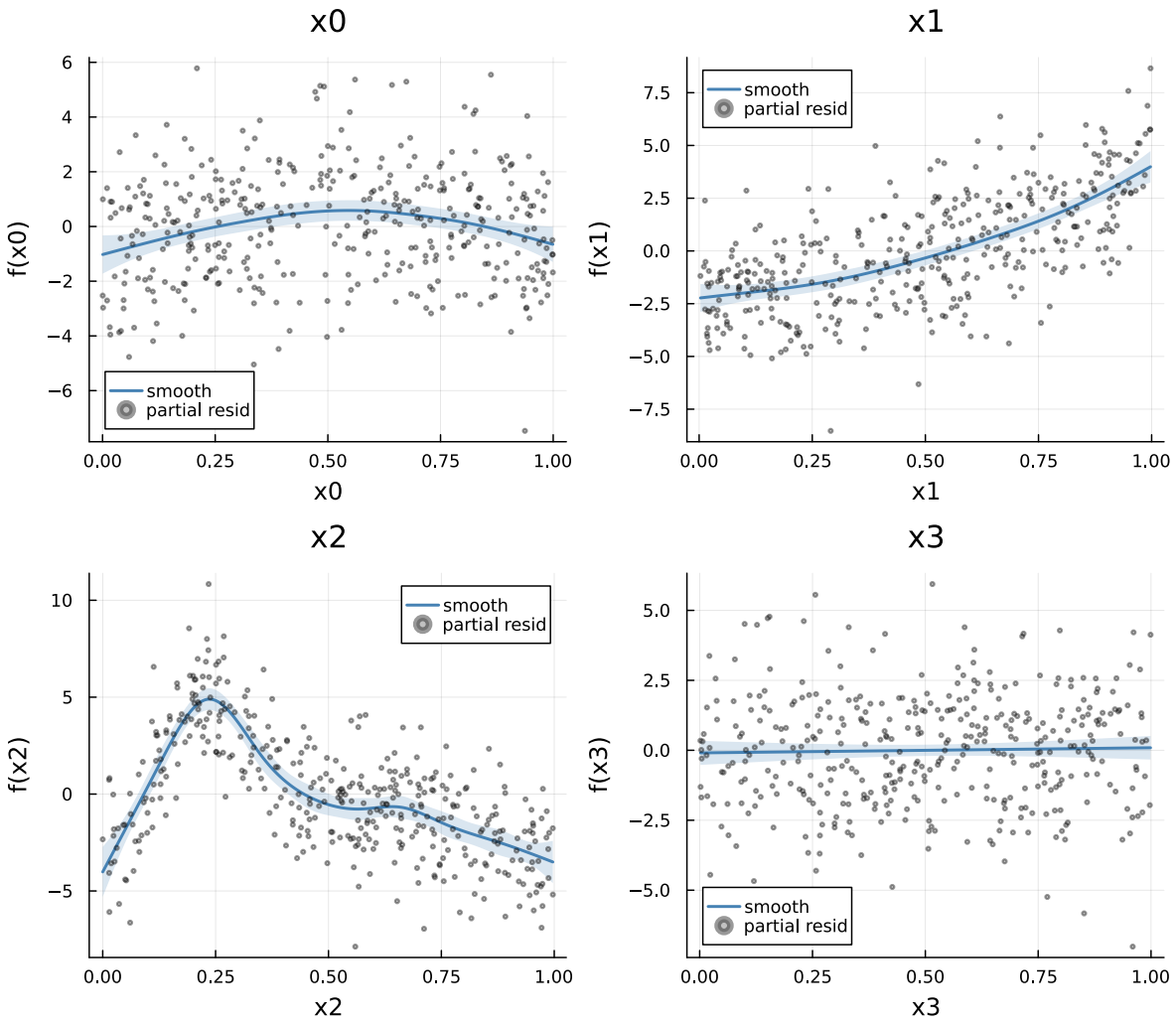
```
se_x0 = smooth_estimates(m; select="s(x0)")
println("Grid points: $(length(se_x0.estimate)), SE range: [$(round(minimum(se_x0.se), digits
```

```
Grid points: 100, SE range: [0.176, 0.35]
```

Partial residuals

`partial_residuals` computes $\hat{f}_j(x_{ij}) + \hat{\varepsilon}_i$ for each smooth, useful for assessing fit quality by overlaying partial residuals on smooth estimates.

```
pr = partial_residuals(m)

plots_pr = []
for (i, xvar) in enumerate(x_vars)
    se_i = smooth_estimates(m; select=string(xvar), n=200)
    xgrid = se_i.covariates[xvar]
    est = se_i.estimate
    se_vals = se_i.se

    label_key = m.smooths[i].spec.label
    xvals, resids = pr[label_key]

    p = plot(xgrid, est;
        ribbon=2 .* se_vals, fillalpha=0.2, color=:steelblue,
        linewidth=2, label="smooth",
        xlabel=string(xvar), ylabel="f($(xvar))",
        title=string(xvar))
    scatter!(p, xvals, resids;
        markersize=1.5, alpha=0.4, color=:grey40, label="partial resid")
    push!(plots_pr, p)
end
plot(plots_pr...; layout=(2, 2), size=(800, 700))
```



Derivatives

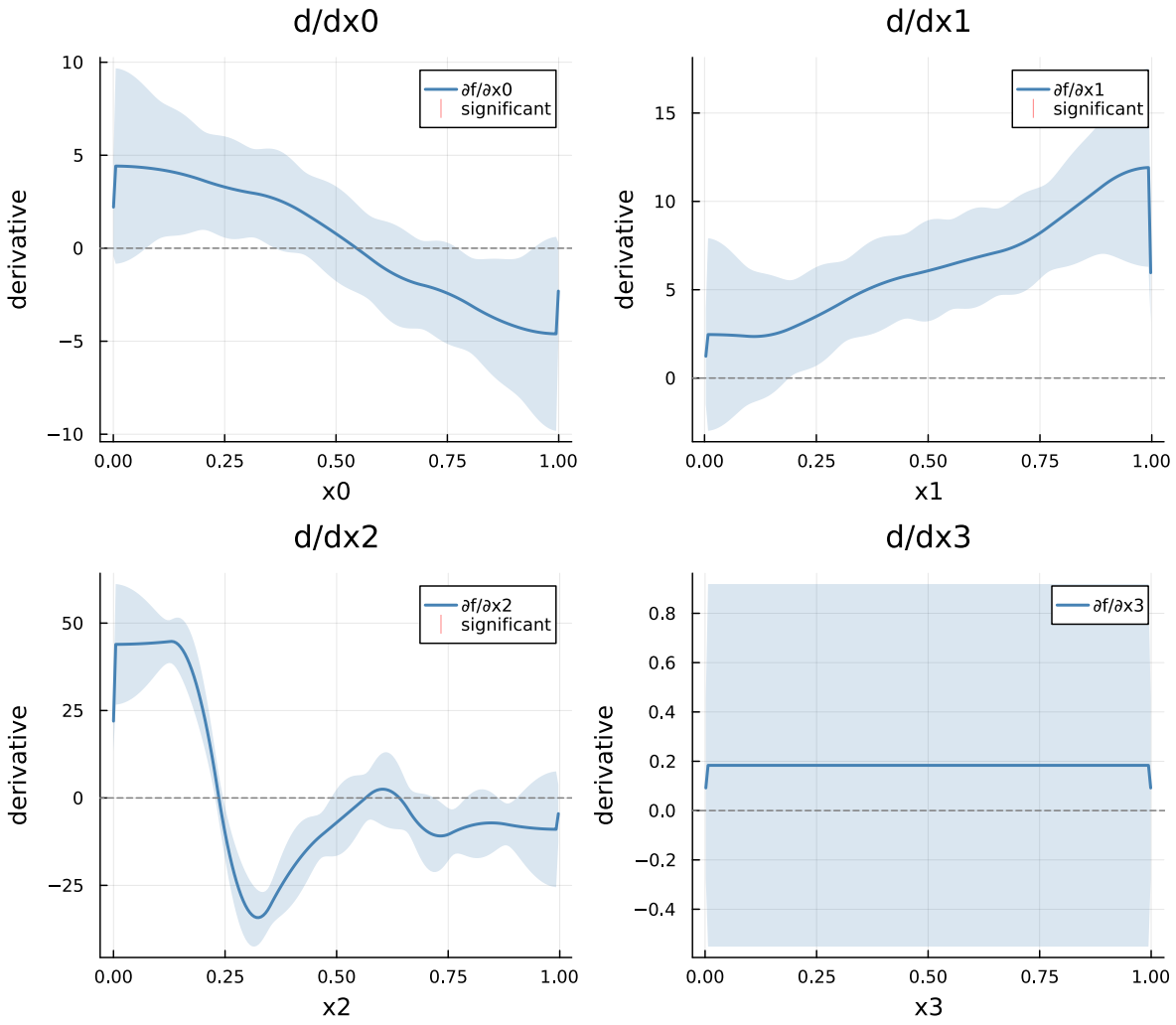
`derivatives` computes finite-difference derivatives of smooth terms with confidence intervals, useful for identifying regions of significant change.

```
d = derivatives(m; order=1, type=:central, n=200, level=0.95)
```

```
DerivativeEstimates (order=1, type=central): 4 smooth(s)
```

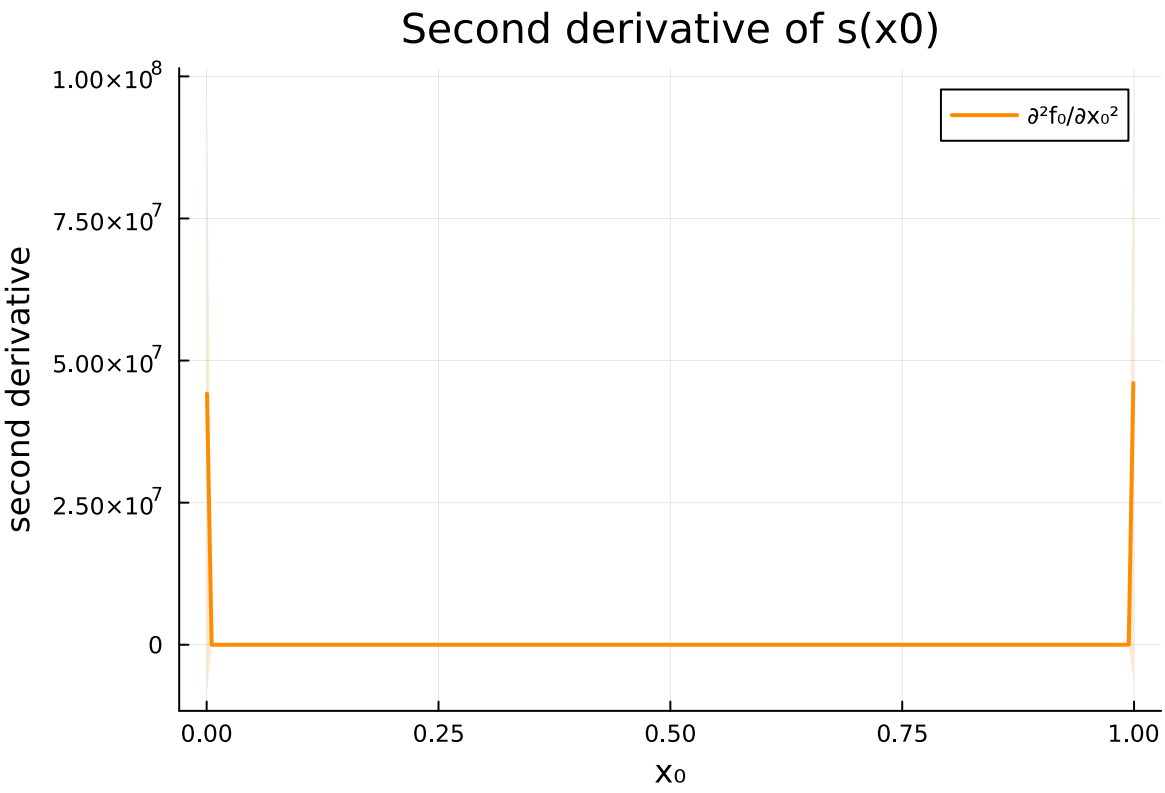
We highlight regions where the confidence interval excludes zero (i.e., statistically significant change):

```
plots_d = []
for (i, xvar) in enumerate(x_vars)
    d_i = derivatives(m; select=string(xvar), order=1, type=:central, n=200)
    xgrid = d_i.x
    deriv = d_i.derivative
    lower = d_i.lower
    upper = d_i.upper

    # Color significant regions
    sig = (lower .> 0) .| (upper .< 0)

    p = plot(xgrid, deriv;
        ribbon=(deriv .- lower, upper .- deriv),
        fillalpha=0.2, color=:steelblue, linewidth=2,
        label="∂f/∂$(xvar)",
        xlabel=string(xvar), ylabel="derivative",
        title="d/d$(xvar)")
    hline!(p, [0.0]; linestyle=:dash, color=:grey50, label="")

    # Mark significant regions with rug marks at bottom
    if any(sig)
        sig_x = xgrid[sig]
        scatter!(p, sig_x, zeros(length(sig_x));
            markersize=2, markerstrokewidth=0, color=:red,
            alpha=0.5, label="significant",
            markershape=:vline)
    end
    push!(plots_d, p)
end
plot(plots_d...; layout=(2, 2), size=(800, 700))
```



Second-order derivatives:

```
d2 = derivatives(m; order=2, select="x0", n=200)
plot(d2.x, d2.derivative;
    ribbon=(d2.derivative .- d2.lower, d2.upper .- d2.derivative),
    fillalpha=0.2, color=:darkorange, linewidth=2,
    label=" ²f / x ²", xlabel="x ", ylabel="second derivative",
    title="Second derivative of s(x0)", size=(600, 400))
```



Model diagnostics with `appraise`

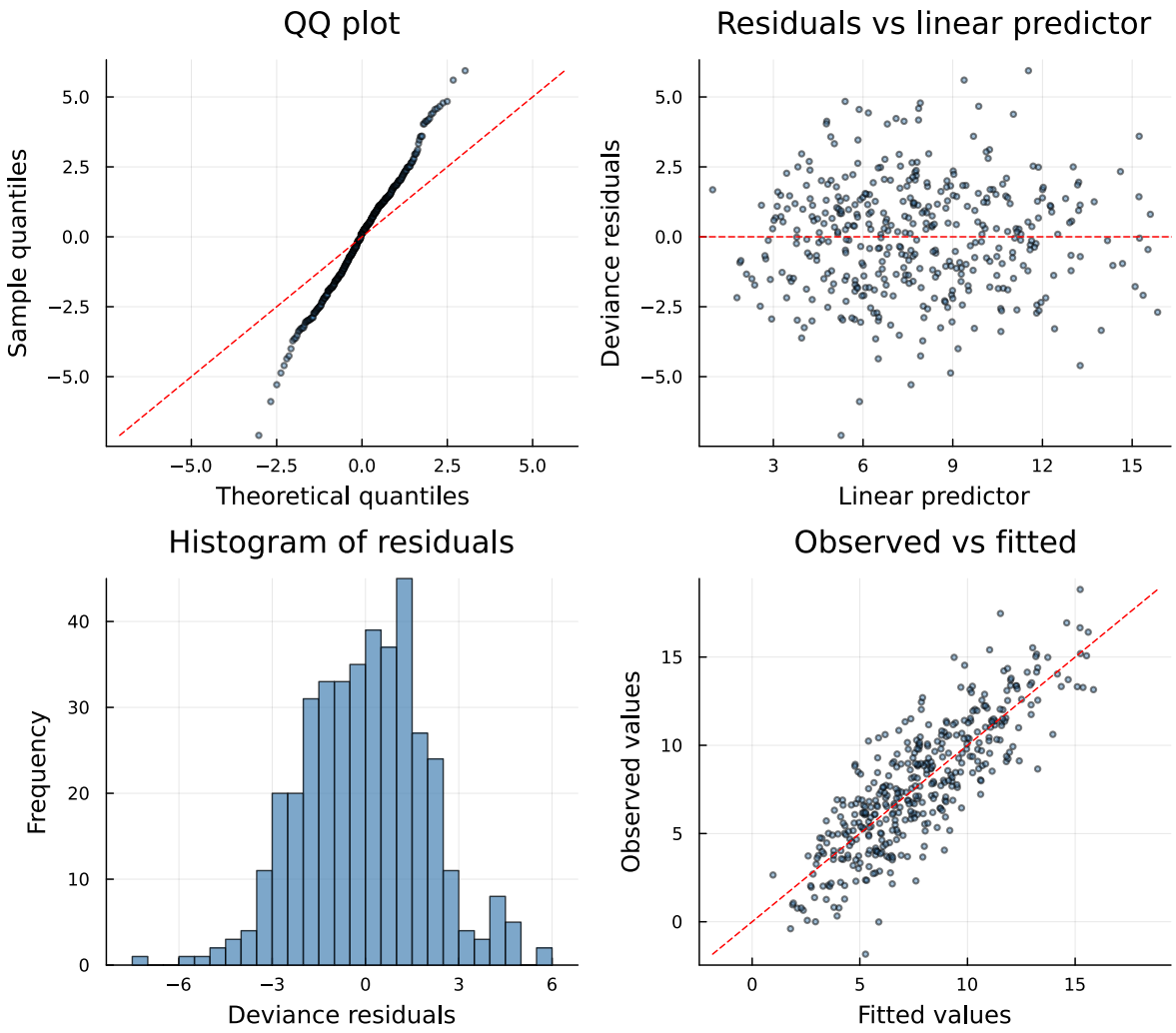
`appraise` returns diagnostic data for the four standard residual plots: QQ plot, residuals vs linear predictor, histogram of residuals, and observed vs fitted values.

```
diag_data = appraise(m)
```

```
AppraiseData: 400 observations
  Deviance residuals: min=-7.100, max=5.939
```

```
p1 = scatter(diag_data.qq_theoretical, diag_data.qq_sample;
    xlabel="Theoretical quantiles", ylabel="Sample quantiles",
    title="QQ plot", label="", markersize=2, alpha=0.5, color=:steelblue)
lims = extrema(vcat(diag_data.qq_theoretical, diag_data.qq_sample))
plot!(p1, [lims[1], lims[2]], [lims[1], lims[2]];
    linestyle=:dash, color=:red, label="")

p2 = scatter(diag_data.linear_predictor, diag_data.residuals_deviance;
```

```
    xlabel="Linear predictor", ylabel="Deviance residuals",
    title="Residuals vs linear predictor", label="",
    markersize=2, alpha=0.5, color=:steelblue)
hline!(p2, [0.0]; linestyle=:dash, color=:red, label="")

p3 = histogram(diag_data.residuals_deviance;
    xlabel="Deviance residuals", ylabel="Frequency",
    title="Histogram of residuals", label="",
    bins=30, color=:steelblue, alpha=0.7)

p4 = scatter(diag_data.fitted, diag_data.observed;
    xlabel="Fitted values", ylabel="Observed values",
    title="Observed vs fitted", label="",
    markersize=2, alpha=0.5, color=:steelblue)
lims4 = extrema(vcat(diag_data.fitted, diag_data.observed))
plot!(p4, [lims4[1], lims4[2]], [lims4[1], lims4[2]];
    linestyle=:dash, color=:red, label="")

plot(p1, p2, p3, p4; layout=(2, 2), size=(800, 700))
```



Posterior simulation

Smooth samples

`smooth_samples` draws posterior realizations of individual smooth functions on an evaluation grid—useful for visualizing uncertainty in smooth shapes.

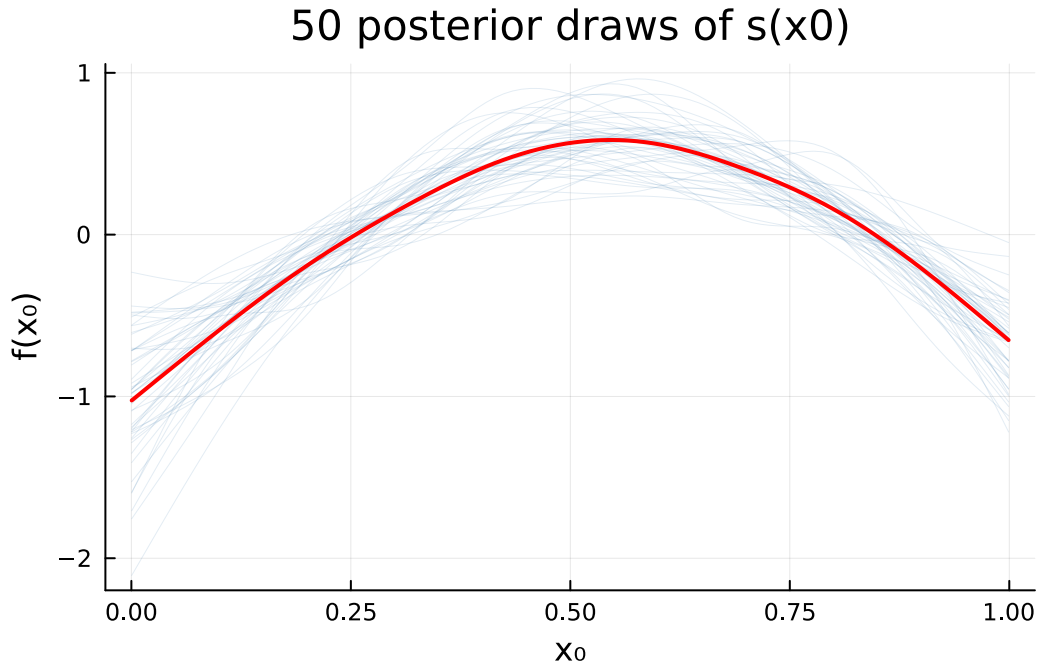
```
ss = smooth_samples(m; n=50, n_grid=100, seed=42)
```

```
Dict{String, Tuple{Vector{Float64}, Matrix{Float64}}} with 4 entries:
  "s(x3,bs=cr)" => ([0.00168691, 0.0117606, 0.0218343, 0.031908, 0.0419817, 0.0…
```

```
  "s(x2,bs=cr)" => ([5.89194e-6, 0.0100758, 0.0201458, 0.0302157, 0.0402856, 0.…
  "s(x1,bs=cr)" => ([0.00289996, 0.0129477, 0.0229955, 0.0330432, 0.0430909, 0.…
  "s(x0,bs=cr)" => ([0.000374096, 0.0104637, 0.0205533, 0.0306428, 0.0407324, 0…
```

We show a "spaghetti plot" of posterior draws for `s(x0)`:

```
label_x0 = m.smooths[1].spec.label
xgrid_ss, smat_ss = ss[label_x0]

p_ss = plot(xlabel="x ", ylabel="f(x )",
    title="50 posterior draws of s(x0)", legend=false)
for j in 1:size(smat_ss, 2)
    plot!(p_ss, xgrid_ss, smat_ss[:, j];
        color=:steelblue, alpha=0.15, linewidth=0.5)
end
# Overlay the point estimate
se_x0 = smooth_estimates(m; select="x0", n=100)
plot!(p_ss, se_x0.covariates[:x0], se_x0.estimate;
    color=:red, linewidth=2)
p_ss
```



Fitted samples

`fitted_samples` draws posterior samples of fitted values $\mathbf{X}\tilde{\beta}$, providing pointwise uncertainty in the mean response.

```
fs = fitted_samples(m; n=200, seed=42)
println("Fitted samples matrix size: ", size(fs))

lower95 = [quantile(fs[i, :], 0.025) for i in 1:size(fs, 1)]
upper95 = [quantile(fs[i, :], 0.975) for i in 1:size(fs, 1)]
println("Median 95% CI width: ", round(median(upper95 .- lower95), digits=3))
```

```
Fitted samples matrix size: (400, 200)
Median 95% CI width: 1.479
```

Concurvity

`concurvity` measures collinearity between smooth terms (the smooth analogue of multicollinearity). Values near 1 indicate high concurvity.

```
conc_full = concurvity(m; full=true)
```

```
4-element Vector{Float64}:
 0.06353085494477784
 0.0631481603048556
 0.059519453582946746
 0.05881844070702802
```

Pairwise concurvity matrix:

```
conc_pair = concurvity(m; full=false)
```

```
4×4 Matrix{Float64}:
 1.0        0.0245173  0.0193346  0.0218418
 0.024944   1.0        0.0186279  0.0200277
 0.021389   0.0199796  1.0        0.0190631
 0.0202408  0.0202776  0.0193991  1.0
```

Basis dimension check

`k_check` assesses whether the basis dimension k is adequate for each smooth. A high EDF-to-k' ratio suggests the basis may be too small.

```
kc = k_check(m)
for item in kc
    println("$(item.label): k' = $(item.k), edf = $(round(item.edf, digits=2)), ratio = $(rou
end
```

```
s(x0,bs=cr): k' = 9, edf = 2.96, ratio = 0.329
s(x1,bs=cr): k' = 9, edf = 3.03, ratio = 0.336
s(x2,bs=cr): k' = 9, edf = 7.82, ratio = 0.869
s(x3,bs=cr): k' = 9, edf = 1.0, ratio = 0.111
```

Summary

GAM.jl provides a comprehensive set of gratia-style diagnostics:

Function	Purpose
`overview`	Tidy summary of smooth terms
`smooth_estimates`	Evaluate smooths on grids with SEs
`derivatives`	Finite-difference derivatives with CIs
`appraise`	Residual diagnostic data
`posterior_samples`	Posterior draws of coefficients
`fitted_samples`	Posterior draws of fitted values
`smooth_samples`	Posterior draws of smooth functions
`partial_residuals`	Partial residuals per smooth
`concurvity`	Smooth collinearity measures
`k_check`	Basis dimension adequacy